sim.py (81줄)

```
#!/usr/bin/env python3
"""RISC-V(RV32I 부분집합) 기능 시뮬레이터 실행 진입점.

사용법:  python sim.py <text.bin 경로> <data.bin 경로>

정상 종료(invalid instruction 을 fetch)
  - 현재 작업 디렉터리에 data.out (0x10000000 부터 정확히 1,024 바이트) 생성
  - 표준 출력에 한 줄:  PC 0x%08x invalid instruction 0x%08x
종료 코드
  0 정상 종료 / 1 인수·파일 입출력 오류 / 2 실행 한도 초과 / 3 메모리 범위·정렬 위반
진단 메시지는 모두 표준 오류(stderr)로만 출력한다.
"""

import sys

from rvsim.cpu import CPU, ExecutionLimitExceeded
from rvsim.memory import DATA_SIZE, Memory, SimError, read_binary_file, write_data_out

EXIT_OK = 0
EXIT_IO_ERROR = 1
EXIT_LIMIT_EXCEEDED = 2
EXIT_RUNTIME_ERROR = 3

DATA_OUT_PATH = "data.out"  # 상대 경로 -> 현재 작업 디렉터리에 생성


def format_termination_line(pc, word):
    """종료 출력 한 줄. 두 숫자 모두 0x 접두사 + 선행 0 포함 소문자 8자리 16진수."""
    return "PC 0x%08x invalid instruction 0x%08x₩n" % (pc, word)


def error(message):
    sys.stderr.write(message + "₩n")


def main(argv):
    if len(argv) != 3:
        error("usage: python sim.py <text.bin> <data.bin>")
        return EXIT_IO_ERROR
    text_path, data_path = argv[1], argv[2]

    try:
        text = read_binary_file(text_path)
        data = read_binary_file(data_path)
    except OSError as e:
        error("error: cannot read input file: %s" % e)
        return EXIT_IO_ERROR

    # 명세의 유효 입력 범위를 벗어난 크기는 경고만 하고 계속 진행한다.
    if len(text) == 0 or len(text) % 4 != 0:
        error("warning: text.bin size %d is not a positive multiple of 4" % len(text))
    if len(data) > DATA_SIZE:
        error("warning: data.bin size %d exceeds %d bytes; extra bytes are ignored"
              % (len(data), DATA_SIZE))

    memory = Memory(text, data)
    cpu = CPU(memory)
    try:
        pc, word = cpu.run()
    except ExecutionLimitExceeded as e:
        error("error: %s (executed %d instructions)" % (e, cpu.executed))
        return EXIT_LIMIT_EXCEEDED
    except SimError as e:
        error("error: %s (executed %d instructions)" % (e, cpu.executed))
        return EXIT_RUNTIME_ERROR

    # 종료 처리: data.out 저장 -> 표준 출력 한 줄
    try:
        write_data_out(DATA_OUT_PATH, memory)
    except OSError as e:
        error("error: cannot write %s: %s" % (DATA_OUT_PATH, e))
        return EXIT_IO_ERROR

    # 텍스트 모드 stdout 은 Windows 에서 "₩n" 을 "₩r₩n" 으로 바꾸므로 바이트로 직접 쓴다.
```

```
75      sys.stdout.buffer.write(format_termination_line(pc, word).encode("ascii"))
76      sys.stdout.buffer.flush()
77      return EXIT_OK
78
79
80  if __name__ == "__main__":
81      sys.exit(main(sys.argv))
```

rvsim/__init__.py (7줄)

```
"""RV32I 부분집합(14개 명령어) 기능 시뮬레이터 패키지.

  bits.py    : 32비트 자르기, 부호 해석, 부호 확장, 비트 필드 추출
  memory.py  : text/data segment, little-endian 읽기·쓰기, 바이너리 파일 입출력
  decoder.py : 명령어 해독, 즉시값 생성, invalid instruction 판별
  cpu.py     : 레지스터·PC, fetch -> decode -> execute 루프, 명령어 실행
"""
```

rvsim/bits.py (53줄)

```
"""32비트 정수 · 부호 처리 도우미.

Python 의 int 는 크기 제한이 없고(임의 정밀도) 음수를 그대로 표현한다.
하드웨어의 32비트 레지스터처럼 동작하게 하려면 결과를 하위 32비트로 잘라야 한다.

이 시뮬레이터의 값 표현 규칙
  - 레지스터 · 메모리에 저장되는 값은 항상 0 ~ 0xFFFFFFFF 범위의
    '부호 없는 32비트 비트 패턴'으로 보관한다.
  - 즉시값(immediate)은 해독 단계에서 부호 확장하여 Python 의 음수/양수 정수로 둔다.
  - 부호 있는 해석이 필요한 곳(blt, bge 비교)에서만 to_signed32() 를 사용한다.
"""

MASK32 = 0xFFFFFFFF


def to_u32(value):
    """임의의 Python 정수를 하위 32비트 비트 패턴(0 ~ 0xFFFFFFFF)으로 자른다.

    예) to_u32(0xFFFFFFFF + 1) -> 0x00000000   (덧셈 결과의 자리올림 버림)
        to_u32(0 - 1)          -> 0xFFFFFFFF   (음수 -> 2의 보수 비트 패턴)
    """
    return value & MASK32


def to_signed32(value):
    """32비트 비트 패턴을 2의 보수 부호 있는 정수(-2^31 ~ 2^31-1)로 해석한다.

    예) 0xFFFFFFFF -> -1, 0x80000000 -> -2147483648, 0x7FFFFFFF -> 2147483647
    """
    value &= MASK32
    if value & 0x80000000:          # bit 31(부호 비트)이 1 이면 음수
        return value - 0x100000000  # 2^32 를 빼서 음수 값으로 바꾼다
    return value


def sign_extend(value, bit_count):
    """bit_count 비트 폭의 값을 부호 확장한다(최상위 비트가 1 이면 음수).

    예) sign_extend(0xFFF, 12) -> -1      sign_extend(0x800, 12) -> -2048
        sign_extend(0x7FF, 12) -> 2047    sign_extend(0x1000, 13) -> -4096
    """
    value &= (1 << bit_count) - 1
    if value & (1 << (bit_count - 1)):
        return value - (1 << bit_count)
    return value


def get_bits(value, hi, lo):
    """value 의 비트 [hi:lo] (양 끝 포함) 필드를 꺼낸다.

    예) get_bits(0x00700113, 6, 0) -> 0x13 (opcode)
    """
    return (value >> lo) & ((1 << (hi - lo + 1)) - 1)
```

rvsim/memory.py (88줄)

```
"""메모리 모델과 바이너리 입출력.

주소 공간 (명세 2절, 6절)
  text segment : 0x00040000 부터 text.bin 크기만큼. 명령어 fetch 전용, 쓰기 불가.
  data segment : 0x10000000 ~ 0x100003FF (1,024 바이트). lw/sw 전용.

바이트 순서: little-endian (명령어·데이터 모두)
  주소 a 부터 네 바이트가 b0 b1 b2 b3 이면
  32비트 값 = b0 + (b1 << 8) + (b2 << 16) + (b3 << 24)
  예) 파일 바이트 78 56 34 12  ->  값 0x12345678
"""

TEXT_BASE = 0x00040000
DATA_BASE = 0x10000000
DATA_SIZE = 1024  # 0x400 바이트: 0x10000000 ~ 0x100003FF


class SimError(Exception):
    """명세의 유효 범위를 벗어난 동작(주소 범위·정렬 위반 등)."""


def read_word_le(buf, offset):
    """buf[offset] 부터 4 바이트를 little-endian 32비트 값으로 읽는다."""
    b0 = buf[offset]
    b1 = buf[offset + 1]
    b2 = buf[offset + 2]
    b3 = buf[offset + 3]
    return b0 | (b1 << 8) | (b2 << 16) | (b3 << 24)


def write_word_le(buf, offset, value):
    """32비트 값을 little-endian 순서(하위 바이트 먼저)로 buf[offset] 부터 4 바이트에 쓴다."""
    buf[offset] = value & 0xFF
    buf[offset + 1] = (value >> 8) & 0xFF
    buf[offset + 2] = (value >> 16) & 0xFF
    buf[offset + 3] = (value >> 24) & 0xFF


class Memory:
    """text segment(읽기 전용)와 data segment(1,024 바이트)를 분리해 관리한다."""

    def __init__(self, text_bytes, data_bytes):
        # text: 수정할 수 없는 bytes 로 보관 -> text segment 쓰기 불가
        self.text = bytes(text_bytes)
        # data: 먼저 1,024 바이트 전체를 0 으로 초기화한 뒤 data.bin 내용을 앞에서부터 복사
        self.data = bytearray(DATA_SIZE)
        n = min(len(data_bytes), DATA_SIZE)
        self.data[0:n] = data_bytes[0:n]

    def fetch(self, addr):
        """명령어 인출: text segment 의 addr 에서 32비트 명령어 값을 읽는다."""
        offset = addr - TEXT_BASE
        if addr % 4 != 0 or offset < 0 or offset + 4 > len(self.text):
            raise SimError("instruction fetch outside text segment or misaligned: "
                           "pc=0x%08x" % addr)
        return read_word_le(self.text, offset)

    def _data_offset(self, addr):
        """data 주소를 self.data 배열의 인덱스로 바꾼다. 4바이트 정렬·범위를 검사한다."""
        offset = addr - DATA_BASE
        if addr % 4 != 0 or offset < 0 or offset + 4 > DATA_SIZE:
            raise SimError("data access outside data segment or misaligned: "
                           "addr=0x%08x" % addr)
        return offset

    def load_word(self, addr):
        """lw: data segment 에서 32비트 값을 little-endian 으로 읽는다."""
        return read_word_le(self.data, self._data_offset(addr))

    def store_word(self, addr, value):
        """sw: data segment 에 32비트 값을 little-endian 으로 쓴다."""
        write_word_le(self.data, self._data_offset(addr), value)


```

```
75 def read_binary_file(path):
76     """입력 파일을 바이너리 모드로 읽는다(ASCII/16진수 텍스트로 해석하지 않음)."""
77     with open(path, "rb") as f:
78         return f.read()
79 
80 
81 def write_data_out(path, memory):
82     """data segment 0x10000000 ~ 0x100003FF 의 1,024 바이트를 주소 증가 순서로 저장한다.
83 
84     "wb" 모드: 기존 파일을 덮어쓰고, 줄바꿈 변환 없이 바이트를 그대로 기록한다.
85     (텍스트 모드로 쓰면 Windows 에서 0x0A 바이트가 0x0D 0x0A 로 바뀌어 크기가 달라진다.)
86     """
87     with open(path, "wb") as f:
88         f.write(bytes(memory.data[0:DATA_SIZE]))
```

rvsim/decoder.py (148줄)

```
"""명령어 해독(decode)과 invalid instruction 판별.

RV32I 32비트 명령어 형식 (비트 번호 31 ~ 0)

        31       25 24   20 19   15 14  12 11         7 6      0
  R형  | funct7    |  rs2  |  rs1  |funct3|     rd     | opcode |
  I형  |     imm[11:0]     |  rs1  |funct3|     rd     | opcode |
  S형  | imm[11:5] |  rs2  |  rs1  |funct3|  imm[4:0]  | opcode |
  B형  |imm[12|10:5]| rs2  |  rs1  |funct3| imm[4:1|11]| opcode |
  U형  |            imm[31:12]            |     rd     | opcode |

지원 명령어 14개와 판별 조건 (이 표에 없는 조합은 모두 invalid instruction)

   명령어 형식  opcode    funct3  funct7
   add    R    0110011   000     0000000
   sub    R    0110011   000     0100000
   and    R    0110011   111     0000000
   or     R    0110011   110     0000000
   addi   I    0010011   000     -
   andi   I    0010011   111     -
   ori    I    0010011   110     -
   lw     I    0000011   010     -
   sw     S    0100011   010     -
   beq    B    1100011   000     -
   bne    B    1100011   001     -
   blt    B    1100011   100     -
   bge    B    1100011   101     -
   lui    U    0110111   -       -
"""

from .bits import get_bits, sign_extend

# opcode (명령어 비트 [6:0])
OPCODE_OP = 0b0110011      # 0x33  R형 연산: add, sub, and, or
OPCODE_OP_IMM = 0b0010011  # 0x13  I형 연산: addi, andi, ori
OPCODE_LOAD = 0b0000011    # 0x03  lw
OPCODE_STORE = 0b0100011   # 0x23  sw
OPCODE_BRANCH = 0b1100011  # 0x63  beq, bne, blt, bge
OPCODE_LUI = 0b0110111     # 0x37  lui

# opcode 별로 funct3(와 funct7) 조합 -> 명령어 이름
R_TYPE_TABLE = {           # (funct3, funct7) -> 이름
    (0b000, 0b0000000): "add",
    (0b000, 0b0100000): "sub",
    (0b111, 0b0000000): "and",
    (0b110, 0b0000000): "or",
}
OP_IMM_TABLE = {0b000: "addi", 0b111: "andi", 0b110: "ori"}
LOAD_TABLE = {0b010: "lw"}
STORE_TABLE = {0b010: "sw"}
BRANCH_TABLE = {0b000: "beq", 0b001: "bne", 0b100: "blt", 0b101: "bge"}


class Instruction:
    """해독된 명령어. 사용하지 않는 필드는 0 이다. imm 은 부호 확장된 Python 정수."""

    __slots__ = ("name", "rd", "rs1", "rs2", "imm")

    def __init__(self, name, rd=0, rs1=0, rs2=0, imm=0):
        self.name = name
        self.rd = rd
        self.rs1 = rs1
        self.rs2 = rs2
        self.imm = imm

    def __repr__(self):
        return "Instruction(%s, rd=%d, rs1=%d, rs2=%d, imm=%d)" % (
            self.name, self.rd, self.rs1, self.rs2, self.imm)


def imm_i(word):
    """I형 즉시값: imm[11:0] = word[31:20], 12비트 부호 확장."""
    return sign_extend(get_bits(word, 31, 20), 12)

```

```
75
76  def imm_s(word):
77      """S형 즉시값: imm[11:5] = word[31:25], imm[4:0] = word[11:7], 12비트 부호 확장."""
78      imm = (get_bits(word, 31, 25) << 5) | get_bits(word, 11, 7)
79      return sign_extend(imm, 12)
80
81
82  def imm_b(word):
83      """B형 즉시값(분기 오프셋, 바이트 단위). 13비트 부호 확장, imm[0] 은 항상 0.
84
85      imm[12]   = word[31]
86      imm[11]   = word[7]
87      imm[10:5] = word[30:25]
88      imm[4:1]  = word[11:8]
89      """
90      imm = ((get_bits(word, 31, 31) << 12)
91             | (get_bits(word, 7, 7) << 11)
92             | (get_bits(word, 30, 25) << 5)
93             | (get_bits(word, 11, 8) << 1))
94      return sign_extend(imm, 13)
95
96
97  def imm_u(word):
98      """U형 즉시값: word[31:12] 를 상위 20비트에 그대로 두고 하위 12비트는 0."""
99      return word & 0xFFFFF000
100
101
102 def decode(word):
103     """32비트 명령어 값을 해독한다.
104
105     지원하는 14개 명령어이면 Instruction 을, 아니면 None(invalid instruction)을 반환한다.
106     opcode 뿐 아니라 funct3, (R형은) funct7 까지 모두 일치해야 지원 명령어로 인정한다.
107     """
108     opcode = get_bits(word, 6, 0)
109     rd = get_bits(word, 11, 7)
110     funct3 = get_bits(word, 14, 12)
111     rs1 = get_bits(word, 19, 15)
112     rs2 = get_bits(word, 24, 20)
113     funct7 = get_bits(word, 31, 25)
114
115     if opcode == OPCODE_OP:                                 # R형
116         name = R_TYPE_TABLE.get((funct3, funct7))
117         if name is None:
118             return None
119         return Instruction(name, rd=rd, rs1=rs1, rs2=rs2)
120
121     if opcode == OPCODE_OP_IMM:                             # I형 연산
122         name = OP_IMM_TABLE.get(funct3)
123         if name is None:
124             return None
125         return Instruction(name, rd=rd, rs1=rs1, imm=imm_i(word))
126
127     if opcode == OPCODE_LOAD:                               # I형 lw
128         name = LOAD_TABLE.get(funct3)
129         if name is None:
130             return None
131         return Instruction(name, rd=rd, rs1=rs1, imm=imm_i(word))
132
133     if opcode == OPCODE_STORE:                              # S형 sw
134         name = STORE_TABLE.get(funct3)
135         if name is None:
136             return None
137         return Instruction(name, rs1=rs1, rs2=rs2, imm=imm_s(word))
138
139     if opcode == OPCODE_BRANCH:                             # B형 분기
140         name = BRANCH_TABLE.get(funct3)
141         if name is None:
142             return None
143         return Instruction(name, rs1=rs1, rs2=rs2, imm=imm_b(word))
144
145     if opcode == OPCODE_LUI:                                # U형 lui
146         return Instruction("lui", rd=rd, imm=imm_u(word))
147
148     return None                                             # 지원하지 않는 opcode
```

rvsim/cpu.py (104줄)

```
"""CPU 상태(PC, 레지스터 x0~x31)와 fetch -> decode -> execute 실행 루프."""

from .bits import to_signed32, to_u32
from .decoder import decode
from .memory import TEXT_BASE

MAX_INSTRUCTIONS = 1000000  # 정상 명령어 최대 실행 개수 (명세 6절)


class ExecutionLimitExceeded(Exception):
    """정상 명령어를 한도만큼 실행한 뒤 다음 fetch 도 지원 명령어인 경우(비정상 종료)."""

    def __init__(self, pc, word):
        super().__init__("execution limit exceeded: pc=0x%08x instruction=0x%08x"
                         % (pc, word))
        self.pc = pc
        self.word = word


class CPU:
    def __init__(self, memory):
        self.memory = memory
        self.regs = [0] * 32   # x0 ~ x31 모두 0 으로 시작 (스택 포인터 초기화 없음)
        self.pc = TEXT_BASE    # 초기 PC = 0x00040000
        self.executed = 0      # 지금까지 실행을 마친 정상 명령어 수

    def write_reg(self, index, value):
        """레지스터 쓰기. x0 는 항상 0 이므로 쓰기를 무시하고, 값은 하위 32비트만 저장한다."""
        if index != 0:
            self.regs[index] = to_u32(value)

    def run(self, limit=MAX_INSTRUCTIONS):
        """invalid instruction 을 fetch 할 때까지 실행한다.

        반환값: (invalid instruction 을 fetch 한 주소, 그 32비트 명령어 값)
        invalid instruction 자체는 실행하지 않는다.
        """
        while True:
            fetch_pc = self.pc                      # fetch 당시 주소를 따로 보존
            word = self.memory.fetch(fetch_pc)      # 1) fetch
            inst = decode(word)                     # 2) decode
            if inst is None:                        # 지원하지 않는 명령어 -> 실행 없이 종료
                return fetch_pc, word
            if self.executed >= limit:              # 이미 한도만큼 실행했는데 또 정상 명령어
                raise ExecutionLimitExceeded(fetch_pc, word)
            self.pc = self.execute(inst, fetch_pc)  # 3) execute + PC 갱신
            self.executed += 1

    def execute(self, inst, pc):
        """명령어 하나를 실행하고 다음 PC 를 반환한다. pc 는 이 명령어의 주소."""
        name = inst.name
        rs1 = self.regs[inst.rs1]   # x0 은 쓰기가 무시되므로 항상 0 이 읽힌다
        rs2 = self.regs[inst.rs2]
        imm = inst.imm              # 부호 확장된 즉시값 (Python 정수, 음수 가능)
        next_pc = pc + 4            # 분기하지 않으면 다음 명령어

        # ---- R형 산술·논리 ----
        if name == "add":
            self.write_reg(inst.rd, rs1 + rs2)         # write_reg 에서 하위 32비트만 저장
        elif name == "sub":
            self.write_reg(inst.rd, rs1 - rs2)         # 음수 결과도 2의 보수 비트 패턴으로
        elif name == "and":
            self.write_reg(inst.rd, rs1 & rs2)
        elif name == "or":
            self.write_reg(inst.rd, rs1 | rs2)

        # ---- I형 산술·논리 (즉시값은 부호 확장된 값) ----
        elif name == "addi":
            self.write_reg(inst.rd, rs1 + imm)
        elif name == "andi":
            self.write_reg(inst.rd, rs1 & to_u32(imm))  # 예) imm=-1 -> 0xFFFFFFFF
        elif name == "ori":
            self.write_reg(inst.rd, rs1 | to_u32(imm))

```

```
        # ---- U형 ----
        elif name == "lui":
            self.write_reg(inst.rd, imm)                # imm = 상위 20비트 << 12

        # ---- 메모리 (유효 주소 = rs1 + 부호 확장 오프셋, 32비트로 자름) ----
        elif name == "lw":
            addr = to_u32(rs1 + imm)
            self.write_reg(inst.rd, self.memory.load_word(addr))
        elif name == "sw":
            addr = to_u32(rs1 + imm)
            self.memory.store_word(addr, rs2)

        # ---- B형 분기 (분기 목표 = 분기 명령어 자신의 주소 + 오프셋) ----
        elif name == "beq":
            if rs1 == rs2:
                next_pc = pc + imm
        elif name == "bne":
            if rs1 != rs2:
                next_pc = pc + imm
        elif name == "blt":                             # 부호 있는 비교
            if to_signed32(rs1) < to_signed32(rs2):
                next_pc = pc + imm
        elif name == "bge":                             # 부호 있는 비교
            if to_signed32(rs1) >= to_signed32(rs2):
                next_pc = pc + imm

        else:
            raise AssertionError("decode() returned unknown instruction: " + name)

        return to_u32(next_pc)
```